

务，因为产生的代码遵循比较规则的模式，而且我们可以做试验，让编译器产生许多不同程序的代码。本章提供了许多示例和大量的练习，来说明汇编语言和编译器的各个不同方面。精通细节是理解更深和更基本概念的先决条件。有人说：“我理解了一般规则，不愿意劳神去学习细节！”他们实际上是在自欺欺人。花时间研究这些示例、完成练习并对照提供的答案来检查你的答案，是非常关键的。

我们的表述基于 x86-64，它是现在笔记本电脑和台式机中最常见处理器的机器语言，也是驱动大型数据中心和超级计算机的最常见处理器的机器语言。这种语言的历史悠久，开始于 Intel 公司 1978 年的第一个 16 位处理器，然后扩展为 32 位，最近又扩展到 64 位。一路以来，逐渐增加了很多特性，以更好地利用已有的半导体技术，以及满足市场需求。这些进步中很多是 Intel 自己驱动的，但它的对手 AMD(Advanced Micro Devices)也作出了重要的贡献。演化的结果是得到一个相当奇特的设计，有些特性只有从历史的观点来看才有意义，它还具有提供后向兼容性的特性，而现代编译器和操作系统早已不再使用这些特性。我们将关注 GCC 和 Linux 使用的那些特性，这样可以避免 x86-64 的大量复杂性和许多隐秘特性。

我们在技术讲解之前，先快速浏览 C 语言、汇编代码以及机器代码之间的关系。然后介绍 x86-64 的细节，从数据的表示和处理以及控制的实现开始。了解如何实现 C 语言中的控制结构，如 if、while 和 switch 语句。之后，我们会讲到过程的实现，包括程序如何维护一个运行栈来支持过程间数据和控制的传递，以及局部变量的存储。接着，我们会考虑在机器级如何实现像数组、结构和联合这样的数据结构。有了这些机器级编程的背景知识，我们会讨论内存访问越界的问题，以及系统容易遭受缓冲区溢出攻击的问题。在这一部分的结尾，我们会给出一些用 GDB 调试器检查机器级程序运行时行为的技巧。本章的最后展示了包含浮点数据和操作的代码的机器程序表示。

网络旁注 ASM:IA32 IA32 编程

IA32，x86-64 的 32 位前身，是 Intel 在 1985 年提出的。几十年来一直是 Intel 的机器语言之选。今天出售的大多数 x86 微处理器，以及这些机器上安装的大多数操作系统，都是为运行 x86-64 设计的。不过，它们也可以向后兼容执行 IA32 程序。所以，很多应用程序还是基于 IA32 的。除此之外，由于硬件或系统软件的限制，许多已有的系统不能够执行 x86-64。IA32 仍然是一种重要的机器语言。学习过 x86-64 会使你很容易地学会 IA32 机器语言。

计算机工业已经完成从 32 位到 64 位机器的过渡。32 位机器只能使用大概 4GB(2^{32} 字节)的随机访问存储器。存储器价格急剧下降，而我们对计算的需求和数据的大小持续增加，超越这个限制既经济上可行又有技术上的需要。当前的 64 位机器能够使用多达 256TB(2^{48} 字节)的内存空间，而且很容易就能扩展至 16EB(2^{64} 字节)。虽然很难想象一台机器需要这么大的内存，但是回想 20 世纪 70 和 80 年代，当 32 位机器开始普及的时候，4GB 的内存看上去也是超级大的。

我们的表述集中于以现代操作系统为目标，编译 C 或类似编程语言时，生成的机器级程序类型。x86-64 有一些特性是为了支持遗留下来的微处理器早期编程风格，在此，我们不试图去描述这些特性，那时候大部分代码都是手工编写的，而程序员还在努力与 16 位机器允许的有限地址空间奋战。

3.1 历史观点

Intel 处理器系列俗称 x86，经历了一个长期的、不断进化的发展过程。开始时，它是第

一代单芯片、16 位微处理器之一，由于当时集成电路技术水平十分有限，其中做了很多妥协。以后，它不断地成长，利用进步的技术满足更高性能和支持更高级操作系统的需求。

以下列举了一些 Intel 处理器的模型，以及它们的一些关键特性，特别是影响机器级编程的特性。我们用实现这些处理器所需要的晶体管数量来说明演变过程的复杂性。其中，“K”表示 1000，“M”表示 1 000 000，而“G”表示 1 000 000 000。

8086(1978 年，29K 个晶体管)。它是第一代单芯片、16 位微处理器之一。8088 是 8086 的一个变种，在 8086 上增加了一个 8 位外部总线，构成最初的 IBM 个人计算机的心脏。IBM 与当时还不强大的微软签订合同，开发 MS-DOS 操作系统。最初的机器型号有 32 768 字节的内存和两个软驱(没有硬盘驱动器)。从体系结构上来说，这些机器只有 655 360 字节的地址空间——地址只有 20 位长(可寻址范围为 1 048 576 字节)，而操作系统保留了 393 216 字节自用。1980 年，Intel 提出了 8087 浮点协处理器(45K 个晶体管)，它与一个 8086 或 8088 处理器一同运行，执行浮点指令。8087 建立了 x86 系列的浮点模型，通常被称为“x87”。

80286(1982 年，134K 个晶体管)。增加了更多的寻址模式(现在已经废弃了)，构成了 IBM PC-AT 个人计算机的基础，这种计算机是 MS Windows 最初的使用平台。

i386(1985 年，275K 个晶体管)。将体系结构扩展到 32 位。增加了平坦寻址模式(flat addressing model)，Linux 和最近版本的 Windows 操作系统都是使用的这种模式。这是 Intel 系列中第一台全面支持 Unix 操作系统的机器。

i486(1989 年，1.2M 个晶体管)。改善了性能，同时将浮点单元集成到了处理器芯片上，但是指令集没有明显的改变。

Pentium(1993 年，3.1M 个晶体管)。改善了性能，不过只对指令集进行了小的扩展。

PentiumPro(1995 年，5.5M 个晶体管)。引入全新的处理器设计，在内部被称为 P6 微体系结构。指令集中增加了一类“条件传送(conditional move)”指令。

Pentium/MMX(1997 年，4.5M 个晶体管)。在 Pentium 处理器中增加了一类新的处理整数向量的指令。每个数据大小可以是 1、2 或 4 字节。每个向量总长 64 位。

Pentium II(1997 年，7M 个晶体管)。P6 微体系结构的延伸。

Pentium III(1999 年，8.2M 个晶体管)。引入了 SSE，这是一类处理整数或浮点数向量的指令。每个数据可以是 1、2 或 4 个字节，打包成 128 位的向量。由于芯片上包括了二级高速缓存，这种芯片后来的版本最多使用了 24M 个晶体管。

Pentium 4(2000 年，42M 个晶体管)。SSE 扩展到了 SSE2，增加了新的数据类型(包括双精度浮点数)，以及针对这些格式的 144 条新指令。有了这些扩展，编译器可以使用 SSE 指令(而不是 x87 指令)，来编译浮点代码。

Pentium 4E(2004 年，125M 个晶体管)。增加了超线程(hyperthreading)，这种技术可以在一个处理器上同时运行两个程序；还增加了 EM64T，它是 Intel 对 AMD 提出的对 IA32 的 64 位扩展的实现，我们称之为 x86-64。

Core 2(2006 年，291M 个晶体管)。回归到类似于 P6 的微体系结构。Intel 的第一个多核微处理器，即多处理器实现在一个芯片上。但不支持超线程。

Core i7, Nehalem(2008 年，781M 个晶体管)。既支持超线程，也有多核，最初的版本支持每个核上执行两个程序，每个芯片上最多四个核。

Core i7, Sandy Bridge(2011 年，1.17G 个晶体管)。引入了 AVX，这是对 SSE 的扩展，支持把数据封装进 256 位的向量。

Core i7, Haswell(2013 年，1.4G 个晶体管)。将 AVX 扩展至 AVX2，增加了更多的

指令和指令格式。

每个后继处理器的设计都是后向兼容的——较早版本上编译的代码可以在较新的处理器上运行。正如我们看到的那样，为了保持这种进化传统，指令集中有许多非常奇怪的东西。Intel 处理器系列有好几个名字，包括 IA32，也就是“Intel 32 位体系结构(Intel Architecture 32-bit)”，以及最新的 Intel64，即 IA32 的 64 位扩展，我们也称为 x86-64。最常用的名字是“x86”，我们用它指代整个系列，也反映了直到 i486 处理器命名的惯例。

旁注 摩尔定律(Moore's Law)

如果我们画出各种不同的 Intel 处理器中晶体管的数量与它们出现的年份之间的图(y 轴为晶体管数量的对数值)，我们能够看出，增长是很显著的。画一条拟合这些数据的线，可以看到晶体管数量以每年大约 37% 的速率增加，也就是说，晶体管数量每 26 个月就会翻一番。在 x86 微处理器的历史上，这种增长已经持续了好几十年。



1965 年，Gordon Moore，Intel 公司的创始人，根据当时的芯片技术(那时他们能够在一个芯片上制造有大约 64 个晶体管的电路)做出推断，预测在未来 10 年，芯片上的晶体管数量每年都会翻一番。这个预测就称为摩尔定律。正如事实证明的那样，他的预测有点乐观，而且短视。在超过 50 年中，半导体工业一直能够使得晶体管数目每 18 个月翻一倍。

对计算机技术的其他方面，也有类似的呈指数增长的情况出现，比如磁盘和半导体存储器的存储容量。这些惊人的增长速度一直是计算机革命的主要驱动力。

这些年来，许多公司生产出了与 Intel 处理器兼容的处理器，能够运行完全相同的机器级程序。其中，领头的是 AMD。数年来，AMD 在技术上紧跟 Intel，执行的市场策略是：生产性能稍低但是价格更便宜的处理器。2002 年，AMD 的处理器变得更加有竞争力，它们率先突破了可商用微处理器的 1GHz 的时钟速度屏障，并且引入了广泛采用的 IA32 的 64 位扩展 x86-64。虽然我们讲的是 Intel 处理器，但是对于其竞争对手生产的与之兼容的处理器来说，这些表述也同样成立。

对于由 GCC 编译器产生的、在 Linux 操作系统平台上运行的程序，感兴趣的人大多并不关心 x86 的复杂性。最初的 8086 提供的内存模型和它在 80286 中的扩展，到 i386 的时候就已经过时了。原来的 x87 浮点指令到引入 SSE2 以后就过时了。虽然在 x86-64 程序中，我们能看到历史发展的痕迹，但 x86 中许多最晦涩难懂的特性已经不会出现了。

3.2 程序编码

假设一个 C 程序，有两个文件 `p1.c` 和 `p2.c`。我们用 Unix 命令行编译这些代码：

```
linux> gcc -Og -o p p1.c p2.c
```

命令 `gcc` 指的就是 GCC C 编译器。因为这是 Linux 上默认的编译器，我们也可以简单地用 `cc` 来启动它。编译选项 `-Og`^① 告诉编译器使用会生成符合原始 C 代码整体结构的机器代码的优化等级。使用较高级别优化产生的代码会严重变形，以至于产生的机器代码和初始源代码之间的关系非常难以理解。因此我们会使用 `-Og` 优化作为学习工具，然后当我们增加优化级别时，再看会发生什么。实际中，从得到的程序的性能考虑，较高级别的优化(例如，以选项 `-O1` 或 `-O2` 指定)被认为是较好的选择。

实际上 `gcc` 命令调用了一整套的程序，将源代码转化成可执行代码。首先，C 预处理器扩展源代码，插入所有用 `#include` 命令指定的文件，并扩展所有用 `#define` 声明指定的宏。其次，编译器产生两个源文件的汇编代码，名字分别为 `p1.s` 和 `p2.s`。接下来，汇编器会将汇编代码转化成二进制目标代码文件 `p1.o` 和 `p2.o`。目标代码是机器代码的一种形式，它包含所有指令的二进制表示，但是还没有填入全局值的地址。最后，链接器将两个目标代码文件与实现库函数(例如 `printf`)的代码合并，并产生最终的可执行代码文件 `p` (由命令行指示符 `-o p` 指定的)。可执行代码是我们要考虑的机器代码的第二种形式，也就是处理器执行的代码格式。我们会在第 7 章更详细地介绍这些不同形式的机器代码之间的关系以及链接的过程。

3.2.1 机器级代码

正如在 1.9.3 节中讲过的那样，计算机系统使用了多种不同形式的抽象，利用更简单的抽象模型来隐藏实现的细节。对于机器级编程来说，其中两种抽象尤为重要。第一种是由指令集体系结构或指令集架构(Instruction Set Architecture, ISA)来定义机器级程序的格式和行为，它定义了处理器状态、指令的格式，以及每条指令对状态的影响。大多数 ISA，包括 `x86-64`，将程序的行为描述成好像每条指令都是按顺序执行的，一条指令结束后，下一条再开始。处理器的硬件远比描述的精细复杂，它们并发地执行许多指令，但是可以采取保证整体行为与 ISA 指定的顺序执行的行为完全一致。第二种抽象是，机器级程序使用的内存地址是虚拟地址，提供的内存模型看上去是一个非常大的字节数组。存储器系统的实际实现是将多个硬件存储器和操作系统软件组合起来，这会在第 9 章中讲到。

在整个编译过程中，编译器会完成大部分的工作，将把用 C 语言提供的相对比较抽象的执行模型表示的程序转化成处理器执行的非常基本的指令。汇编代码表示非常接近于机器代码。与机器代码的二进制格式相比，汇编代码的主要特点是它用可读性更好的文本格式表示。能够理解汇编代码以及它与原始 C 代码的联系，是理解计算机如何执行程序的关键一步。

`x86-64` 的机器代码和原始的 C 代码差别非常大。一些通常对 C 语言程序员隐藏的处理器状态都是可见的：

- 程序计数器(通常称为“PC”，在 `x86-64` 中用 `%rip` 表示)给出将要执行的下一条指令在内存中的地址。

① GCC 版本 4.8 引入了这个优化等级。较早的 GCC 版本和其他一些非 GNU 编译器不认识这个选项。对这样一些编译器，使用一级优化(由命令行标志 `-O1` 指定)可能是最好的选择，生成的代码能够符合原始程序的结构。